A separate word known as the *token* is mapped to the documents carrying that word. Therefore, when we are searching for a document with a particular word or phrase in our data, we don't have to search the whole document. Instead, indexes will directly give the document associated with the desired word or phrase.

Indexed documents are then sent into a buffer, and wait for a certain period to get stored as segments. Segment data is divided and stored into small blocks called shards. This reduces the time taken for the document to become available for search after the process of indexing. In versions before 6.7.0, Elasticsearch is used to directly store to disks using the *fsync()* function, which is very costly as it involves directly writing to the disk. In Elasticsearch 6.7.0, the latest version, this has been changed to a file cache system that stores data in a buffer first, and then commits it to the disk, increasing the performance.

Tokens	Analysis	Tokens
Swimming		swim
Dance		dance
The	After analysis	sad
Sad		
Swimmer		
Upset		

Table 2

As can be seen in Table 2, after analysis, stop words (like 'the') are removed, synonyms combined (like 'sad'and 'upset') and stemming is done ('swimming' and 'swimmer' to 'swim').

Shard is a low-level worker unit that holds just a slice of all the data. Index, as mentioned above, is just a logical name space that points to one or more shards. Elasticsearch distributes the given data in clusters by storing them as shards, which are then allocated to nodes present in the cluster. Elasticsearch needs to find the shard where it has saved a particular document. Therefore, the process should be deterministic and can be described as follows:

```
shard number = Hash( _id ) % number_of_
primary_shards
```

After fixing the shard, we have a way to find the location of any document. This means that any node in the cluster knows about every document and can forward the call to the correct node.

Under the hood

The Lucene engine stores data in shards, i.e., data blocks. A node can have many shards but this creates a big, single point of failure. Elasticsearch handles failure by doing replication across the cluster. We have to mention the number of primary index shards at the time of indexing. This fixes the number of primary nodes, and helps to coordinate in finding the location of appropriate nodes.

Let us say that we have two primary indexes and a two-node architecture.

The master node finds the shard containing the required documents (here, Node 2) and forwards the request to the corresponding node, as shown in Figure 6.

The effect of replication on queries

- *Insert:* We send the request to the master node (each cluster has one), which finds the shard containing the document with

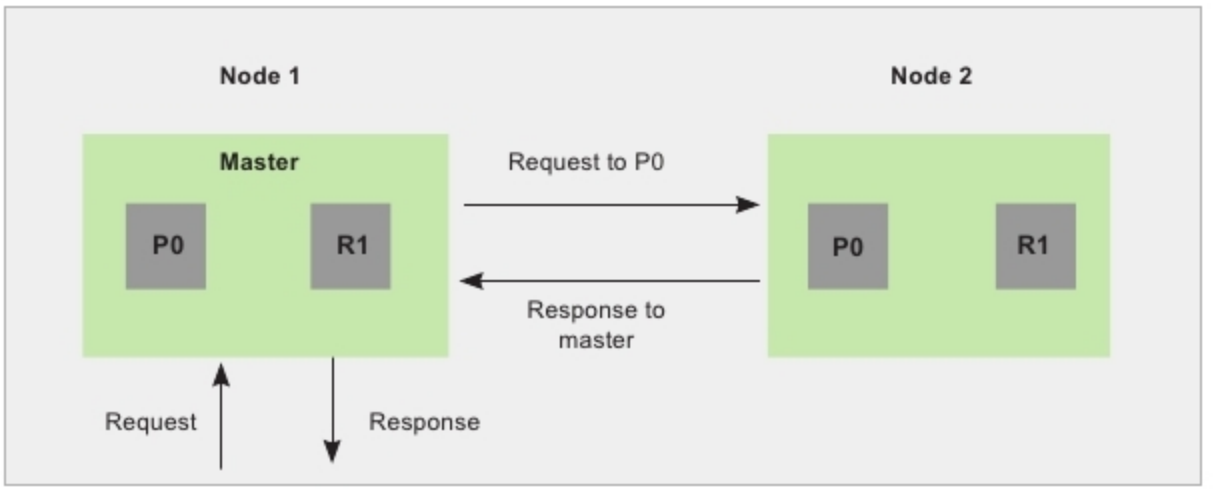

Figure 6: The master node finds the shard containing the required documents, and sends the request to the corresponding node

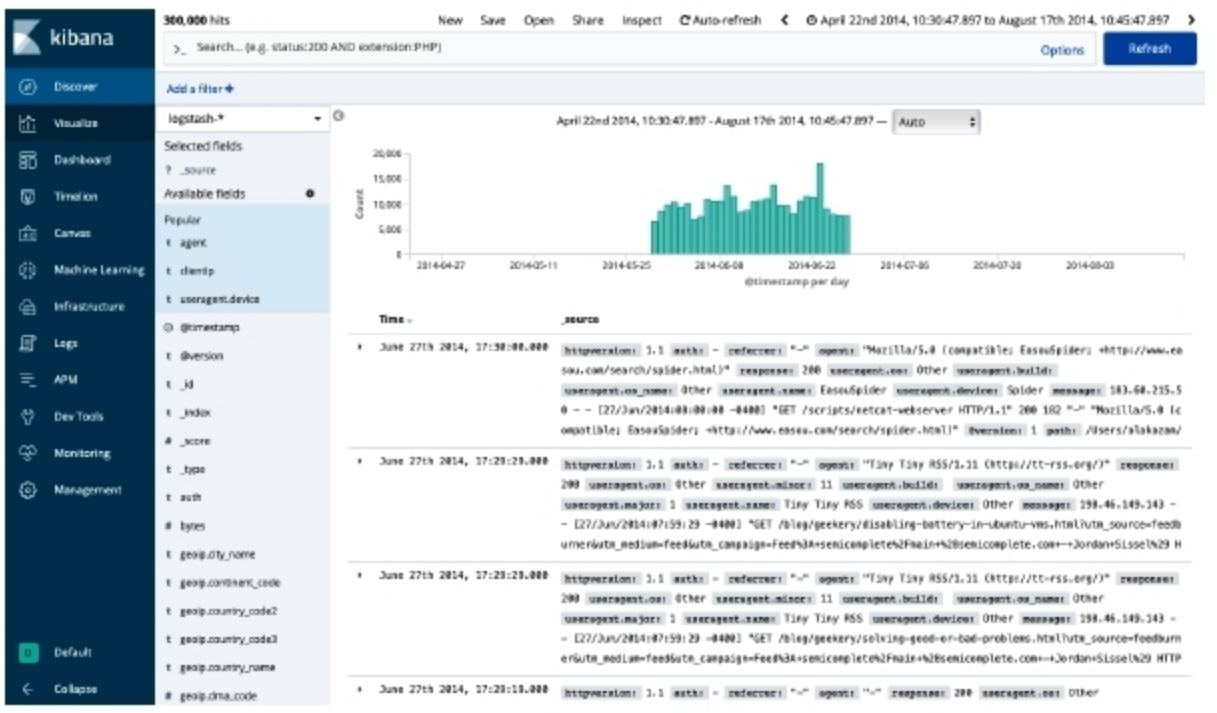

Figure 7: Elasticsearch data stores being visualised in Kibana